

MTE544 – Autonomous Mobile Robotics

University of Waterloo

Department of Mechanical and Mechatronics Engineering

Lab 2: Closed loop control of mobile robots

Group member (Family Name, First Name)	**Student ID**
Adams, Hunter	20997503
Chen, Joe	20998545
Wielowieyski, Jacob	21004619

Station number: _____

Robot number: various

November 4, 2026

1 Logged errors, tuning process and trajectories

Setup. TurtleBot 4, internal odometry as the position sensor, 10 Hz control loop. The distance error e_d is the distance to the goal and the heading error e_θ is the bearing to the goal minus θ, wrapped to $[-\pi, \pi]$; $\dot{e}$ is the slope over the last three samples and $\int e$ uses the trapezoid rule on the odometry time stamps. Commands are limited to 0.22 m/s and 1.9 rad/s. For the point controller we deliberately kept the gains low enough that the commands stay below these limits, so that the linear behaviour of each controller can be observed; a saturated controller only shows the limit. Goal tolerance: 1 cm for point runs, 5 cm for trajectories. Each run is named by its start time (e.g. 17:12:26). Low batteries cost us lab time: we changed robots several times and waited for charging. Due to limited time, the TAs told us to skip PID point runs, so the point-controller results on the robot are for P; Fig. 1 adds a simulated PID run, and Section 2 discusses how we expect PID to behave.

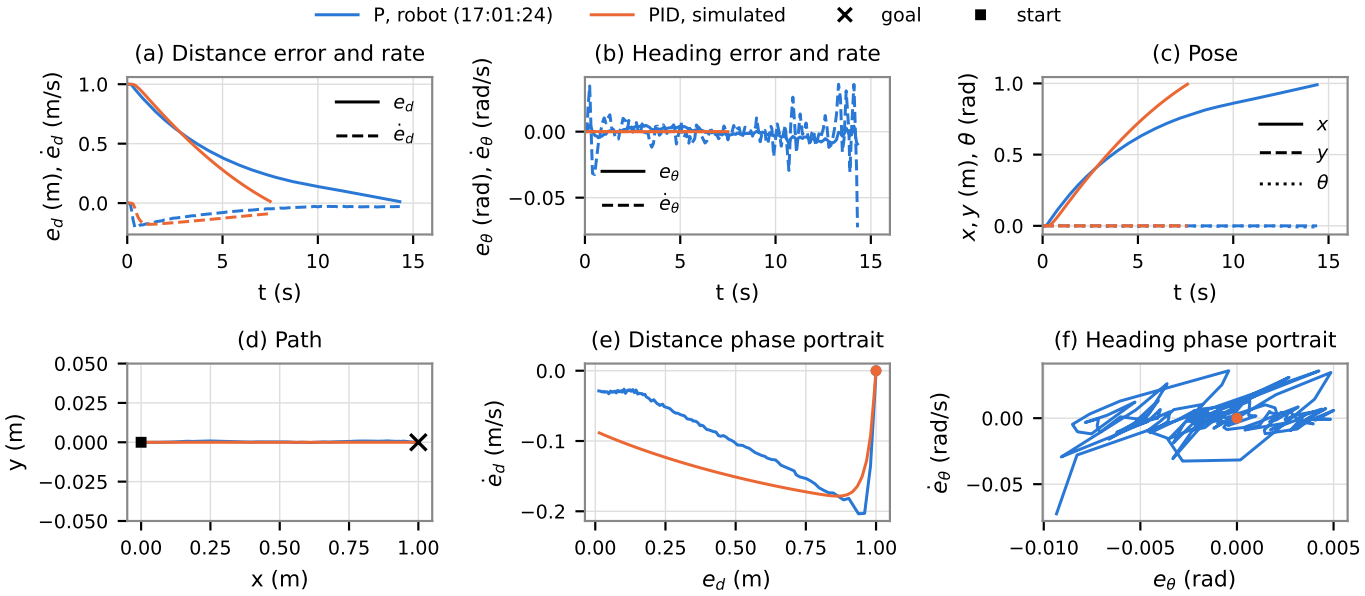


Figure 1: Point controller, goal 1 m straight ahead, $k_{lp} = 0.2$ (heading error below 0.6°, so the heading gains play no part). Blue: P on the robot (17:01:24). Orange: PID, same controller code, gains of our last robot runs ($k_{lv} = 0.05$, $k_{li} = 0.025$, $k_{ap} = 1.0$, $k_{av} = 0.2$, $k_{ai} = 0.025$; 18:17:44–18:27:45), **simulated, not measured**, on a robot model fitted to run 17:12:26 (`sim_point.py`). (a, b) e and $\dot{e}$ vs time, (c) x, y, θ vs time, (d) path, (e, f) $\dot{e}$ vs e. Neither command reaches the 0.22 m/s limit (PID peaks at 0.207 m/s). P slows in proportion to e_d and stops 9 mm from the goal at 0.03 m/s after 14.4 s; the integral keeps PID faster: within 1 cm after 7.6 s, still at 0.09 m/s, so after the stop command it rolls ≈4 cm past the goal (P: ≈1.5 cm; model delay + lag, after the logs end). With a goal behind the robot the distance integral winds up during the turn and the simulated PID circles the goal, so we avoided it.

Tuning process (Fig. 2). *(a) Distance gain.* When driving 1 m straight ahead with unsaturated gains, e_d decays exponentially as predicted by a theoretical first-order model: fitted time constants 9.8 s at $k_{lp} = 0.1$ and 5.0 s at 0.2 ($1/k_{lp} = 10$ and 5 s). The last part of the graph turns into a straight line because the robot does not drive slower than about 0.03 m/s (lower end saturation is hit). To find where upper saturation starts we raised k_{lp}: from 0.3 the command starts at the limit and quickly drops, and from 2.5 it stays there until the last 9 cm (4.8 s to the goal whatever the gain). We chose $k_{lp} = 0.2$, the largest gain that keeps $v = k_{lp} e_d$ below 0.22 m/s for our 1 m goals. *(b) Heading gain, goal behind the robot.* For a 180° turn, $\omega = k_{ap} e_\theta$ stays below 1.9 rad/s only for $k_{ap} < 0.6$. With 0.4 and 0.5 the heading decays without overshoot (fitted time constants 4.1 and 2.6 s vs $1/k_{ap} = 2.5$ and 2 s; slower than the ideal loop because the bearing to the goal keeps moving while the robot drives sideways). This is because a lower k_{ap} drives a wider arc (627 and 491 mm), since the distance loop drives forward during the turn. A saturated run ($k_{ap} = 2.5$) turns faster (1.15 s rise, 196 mm arc) but overshoots and hides the linear dynamics. We chose $k_{ap} = 0.5$ (run 17:12:26). *(c) Steering gains, PID on the parabola.* The steering looked choppy, so in the limited time we focused on the

angular loop; the speed command sat at its 0.22 m/s limit in every PID run because of k_{li}; the turn command never hit saturation. At $k_{ap} = 0.5$ the RMS heading error was 15–16° and the robot strayed up to 83–95 mm from the path; $k_{ap} = 1.0$ halved the heading error to 8° and the worst distance from the path to 34–38 mm, and raising k_{av} to 0.5 gave our best parabola run (29 mm worst, 13.2 s). Its turn command was choppier ($\dot{e_\theta}$ jumps at each waypoint change), but the measured turn-rate jitter only rose from 0.020 to 0.026 rad/s. The runs after it ($k_{ap} = 0.5$, then $k_{av} = 0.2$) were checks of nearby gains in the remaining lab time and did not beat it.

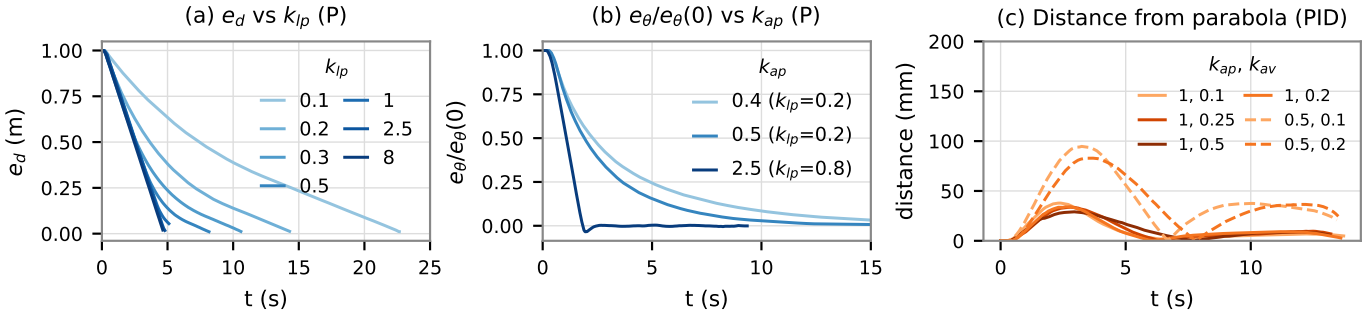


Figure 2: Tuning on the robot. (a) e_d for seven values of k_{lp} (P, goal 1 m ahead). (b) Normalised heading error for three values of k_{ap} (P, goal behind; the 2.5 run used $k_{lp} = 0.8$, the others 0.2). (c) Distance from the parabola for six PID gain sets ($k_{lp} = 0.2$, $k_{li} = k_{ai} = 0.025$).

Trajectories and choice of controller (Fig. 3). Most of our trajectory control tuning time went into the parabola (16 runs). We chose a PID with the following: ($k_{lp} = 0.2$, $k_{li} = 0.025$, $k_{ap} = 1.0$, $k_{av} = 0.5$, $k_{ai} = 0.025$). With the lab time left, the sigmoid was run with the gains of our last parabola check ($k_{av} = 0.2$, 34 mm worst on the parabola). Because e_d is measured to the end of the curve, P slows down over the last $v_{\max}/k_{lp} = 1.1$ m. With the same k_{lp} and k_{ap}, P, PI and PID tracked the parabola similarly well (RMS 13–15 mm, worst error in the tight first 0.5 m), but the integral term cut the time to come within 5 cm of the end from 21.3 s to 13.2–13.3 s (partially because it saturated the velocity, but it would still be expected to improve performance as it would minimize slowdown as the robot got nearer the target), and the D term gave the lowest worst-case deviation of the three (29 mm). With a 1 cm tolerance the PI controller circled the end of the parabola (17:49:49, 17:53:25): the integral keeps $v = 0.22$ m/s, and the tightest turn at that speed, $v_{\max}/\omega_{\max} = 0.12$ m, cannot hit a 1 cm circle, so the trajectory tolerance was set to 5 cm. On the sigmoid the worst error (89 mm) is at the start, where the curve leaves the origin at 45° while the robot faces $+x$; on a stretched sigmoid that starts at 27° the RMS error fell from 37 to 20 mm. This is because the heading of the robot does not initially align with the trajectory, and the robot drives forward while turning.

2 P vs PID point controller: stability analysis and discussion

Stability analysis. *Distance loop.* With the heading aligned, $\dot{e_d} = -v$, so the robot itself is an integrator. P control gives $\dot{e_d} = -k_{lp} e_d$: first order, time constant $1/k_{lp}$, no overshoot and zero steady-state error to a fixed goal. Our unsaturated runs closely match: fitted time constants 9.8, 5.0 and 3.2 s for $k_{lp} = 0.1$, 0.2 and 0.3 (theoretical predicted 10, 5 and 3.3 s). *Heading loop.* The robot responds to a step command 0.2–0.4 s later; modelling this as a lag $\tau \approx 0.2$ s gives $\tau\ddot{e_\theta} + (1 + k_{av})\dot{e_\theta} + k_{ap} e_\theta = 0$ and $\zeta = (1 + k_{av})/(2\sqrt{k_{ap}\tau})$. Our P gains $k_{ap} = 0.4$–0.5 give $\zeta = 1.8$–1.6: overdamped, and we measured no overshoot. As a check on τ, the saturated $k_{ap} = 2.5$ run ($\zeta = 0.71$, predicted 4.3 %) overshot by 3.4 %. *Expected PID behavior.* With an I term the distance loop becomes $(1 + k_{lv})s^2 + k_{lp}s + k_{li} = 0$: $\zeta = 0.31$, so our default $k_{li} = 0.1$ turns an overdamped loop into an oscillatory one. e_d would reach zero after ≈4.3 s while still moving at the 0.22 m/s limit, and overshoot by ≈50 % in the linear loop; $\zeta \geq 0.7$ needs $k_{li} \leq 0.02$. The heading loop $(1 + k_{av})s^2 + k_{ap}s + k_{ai} = 0$ has roots −0.33 and $-0.07\,\mathrm{s}^{-1}$: the fast one gives about the same turn as P (rise ≈3.8 s), the slow one, from the integral, makes e_θ cross zero at ≈5.7 s, overshoot by ≈10 % (≈18° on a 180° turn) and creep back with a 14 s time constant. Just adding D would only add damping to a loop that is already overdamped (ζ 1.6 →

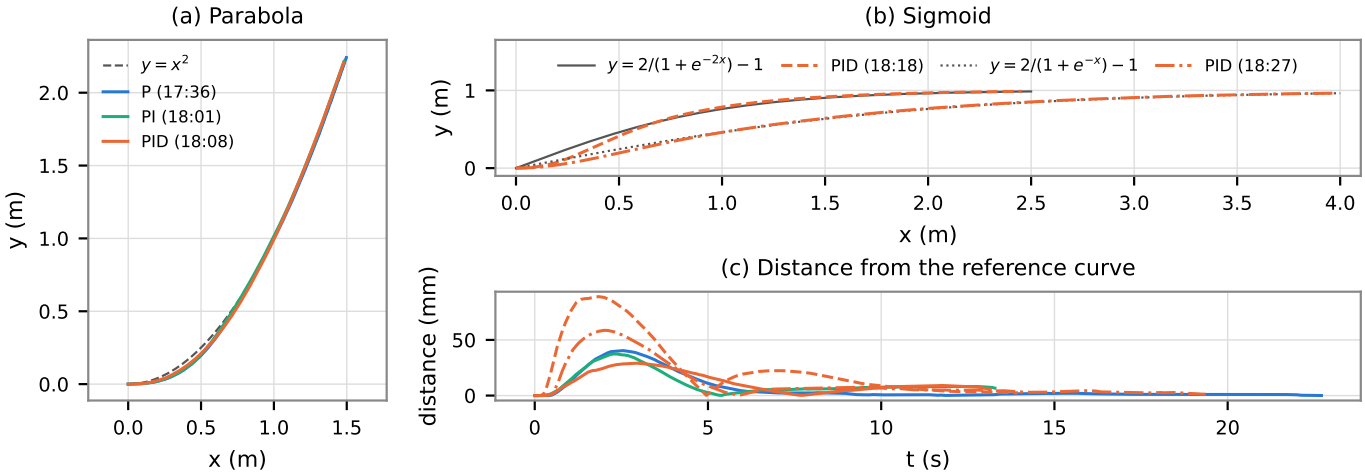


Figure 3: Trajectories on the robot. (a) Parabola $y = x^2$, $x \in [0, 1.5]$, with P (17:36:05), PI (18:01:56) and our chosen PID (18:08:53) at the same $k_{lp} = 0.2$, $k_{ap} = 1.0$. (b) Sigmoid $y = 2/(1 + e^{-2x}) - 1$, $x \in [0, 2.5]$, with PID at $k_{av} = 0.2$ (18:18:30), and the stretched sigmoid $y = 2/(1 + e^{-x}) - 1$, $x \in [0, 4]$, with the same gains (18:27:45). (c) Distance from the robot to the reference curve, line styles as in (a, b). The P run used a 1 cm end tolerance (PI, PID: 5 cm); it was within 5 cm of the end after 21.3 s.

Table 1: Point controller, goal 1 m behind the robot, $k_{lp} = 0.2$, $k_{ap} = 0.5$. P: measured on the robot (run 17:12:26). **PID: expected, not measured** (PID point runs were skipped on the TAs' recommendation due to limited lab time), with our default $k_{lv} = 0.05$, $k_{li} = 0.1$, $k_{av} = 0.25$, $k_{ai} = 0.03$, from the linear analysis of each loop below (lag $\tau = 0.2$ s). Rise time: error from 90 % to 10 %. Settling: last exit from ±5 %. Heading metrics leave out the last 15 cm, where the bearing is ill-defined.

	Metric	P, measured	PID, expected
Agility	Heading rise time (s)	5.6	≈3.8 (same analysis gives 4.0 for P)
	Heading settling time, 5 % (s)	8.3	≈24 (slow integral mode, ≈14 s)
	Distance rise time (s)	10.9	≈3.2
	Time to goal (s)	18.0	≈4.3 to first reach it, at 0.22 m/s
Accuracy	Final position error (mm)	8.1	same if it stops on the first pass
	Steady-state error	0 (robot integrates v)	0, I term not needed
Overshoot	Heading overshoot (%)	0	≈10 (≈18° at ≈11 s)
	Distance overshoot (%)	0	≈50 in the linear loop

2.0), slowing the turn slightly.

Discussion. *Agility:* the I term would make the distance loop much faster (e_d reaching zero after ≈4.3 vs 18 s), while the turn would take about as long (heading rise ≈3.8 vs 4.0 s in the same analysis). When comparing the same effect on the parabola in lab, the I term cut the time from 21.3 to 13.2 s. *Accuracy:* the robot integrates the speed command; since P already has zero steady-state error (it stopped 8 mm from the goal), an I term adds no accuracy for a fixed goal. *Overshoot:* PID is expected to perform worse: ≈10 % heading overshoot with a slow tail, and arrival at full speed, where at 22 mm per control period the robot can step over the 1 cm goal circle and orbit it, as our PI parabola runs did with a 1 cm tolerance. It is possible that a very low I term would lessen this effect.

For a fixed goal we would therefore keep P, add D only if a higher k_{ap} made the heading underdamped (above $k_{ap} \approx 1.25$ for $\tau = 0.2$ s), and use an I term only with $k_{li} \leq 0.02$ and a clamp. For trajectories, the wide arc at the start comes from driving while turning; scaling the speed by $\cos e_\theta$ would remove it. P with $k_{ap} = 3.0$ tracked tightest (8 mm, 23.1 s), so a higher k_{ap} is the next PID tuning step.